SOFTWARE DESIGN DESCRIPTION

Rat Race Game Description

By: Ryan Mendez & An Nguyen

CPSC 490

Professor: Dr. Lidia Morrison

Department of Computer Science

California State University, Fullerton

Table of Contents

1. INTRODUCTION **1**

1.1 Purpose 2

1.2 Scope 2

1.3 Overview 2

2. FUNCTIONAL REQUIREMENTS **3-6**

3. SYSTEM ARCHITECTURE **8**

4. UML Diagrams **9**

4.1 Use Case Diagram 10

4.2 Sequence Diagram 11-13

4.3 Activity Diagram 14-15

4.5 Class Diagram 16

Table of Figures

Figure No.	Name	Page No.
Figure 3.1	Client-Server Architecture Diagram……..	9
Figure 4.1	Use Case Diagram for Rat Race Escape……..	10
Figure 4.2	Simplified Class Diagram for Rat Race Escape ……...	11-13
Figure 4.3	High-Level Game Flow Activity Diagram……..	15
Figure 4.4	Sequence Diagram for Player Movement and State Update…….	16

1. INTRODUCTION

1.1 Purpose

The goal of the 'Rat Race Escape' is to deploy a fun, entertaining, and reachable 2D asymmetrical multiplayer game. This paper outlines the system requirements and specifications of the web application RAT race escape.

1.2 Scope

The web application will enable the connection of 2-5 players and play a game of cat and mouse in real time. The play is centered on one player (the Cat) preying on the rest (the Rats), which in turn have to cooperate to find the solution and run. The application does user authentication, real-time syncing of the game state, and offers a smooth, browser-based interface, without downloads.

1.3 Overview

The other parts of this paper discuss the functional and non-functional set of requirements in creating the Rat Race Escape game. Section 2 describes the functional requirements as viewed by the user and Section 3 contains the description of the non-functional requirements.

2. FUNCTIONAL REQUIREMENTS

1. The application will have a starting page.
   - Description: Once a user lands on the URL of the game, he/she will see a principal menu.
   - Precondition: The customer should possess a valid URL and a compatible web browser.
   - Postcondition: The main menu appears with the possibility of choosing between the options of:Login,register and settings.

2. The application will enable creation of an account by a new user.
   - Description: A new user will be able to register with the help of a unique username, email, and password.
   - Precondition: The email and the username should not be used already.

- Post condition: A new user account is established and is registered in the database.

3. The application will show a multiplayer game lobby.
   - Description: Lobby will display the game rooms available, number of players and enable the user to form or join rooms.
   - Precondition: The user should have been logged in.
   - Post condition: The user is able to view and participate in available game sessions.

4. The app will enable a user to form a new game room.
   - Description: A user is able to create a room, and set its parameters such as the choice of the map, and the number of players (2-5).
   - Precondition: It should be in the lobby of the user.
   - Post condition: Another room is created and can be seen by other players in the lobby.

5. The app will enable one to join a game room that is already in progress.
   - Description: A user will have the opportunity to choose and be part of an available room that is not occupied.
   - Conditions: The user should be located in the lobby and the room should have a slot available.
   - Postcondition The user is added to the player list of the room.

6. The server will automatically initiate the game by having 2-5 players in a room.
   - Description: When the minimum number of players (2) is reached, and the creator initiates the game, a countdown timer will be started, and then, a game session will be launched.
   - Condition: There should be 2-5 players in a room.
   - Postcondition: The game session begins and all players are changed into the game screen.

7. The server will then randomly give one of the players the role of 'Cat' and the other players the role of Rats.
   - Description: Roles are given fairly by randomly selecting one to play at the beginning of each game.
   - Precondition: The game session is initiated.

- Postcondition: Everyone knows his or her role.

8. The server will come up with a game map that will contain obstacles, concealed areas, and interactive escapes.

- Description: The game world is either procedurally generated or chosen in the selection of pre-designed maps.
- Precondition: The game session was initiated.
- Postcondition: To all clients, a playable map is generated and in its initial state is given.

9. The client will send back the game state that is obtained by the server.

- Description: The client shows the map, all the characters in the game (Cat and Rats), and the interactivity.
- Precondition: The client has been issued with the initial game state by the server.
- Post condition: The player is able to observe the game world, and the character.

10. The software will enable the users to transmit movement and action instructions to the server.

- Description: The input devices (keyboard/mouse) are used to move players and interact with the objects and elements (e.g., levers, doors).
- Precondition: The game is going on and the player has not been killed.
- Postcondition: The action that the player is supposed to take is relayed to the game server to process.

11. The server will authenticate any actions of the players and change the state of the game.

- Description: The server is the game logic authority, which validates moves, collisions, and interactions.
- Precondition: A client has received an action command.
- Postcondition: A game state (e.g. position of player) changed.

12. The server will disseminate the updated game state to all the clients connected to it periodically.

- Description: State updates are propagated in the server with a frequency of several Hz.

- Precondition: The state of the game has been modified.
- Postcondition: The current game state is given to all clients and reset their view.

13. The catch mechanic will be met in the application when the Cat hits a Rat.
    - Description: When the server recognizes a collision between the Cat and a Rat, the Rat is taken and eliminated in the round.
    - Precondition: The Map has the Cat and a Rat close to one another.
    - Post condition: The character of the caught Rat is paralyzed/eliminated and all the players are informed.

14. The application will deal with the 'catch' mechanic when the Cat hits a Rat.
    - Description: When the server notices that the Cat and a Rat have collided, the Rat is picked and eliminated off the round.
    - Precondition: There is a close distance between the Cat and a Rat on the map.
    - Postcondition: The character of the caught Rat is immobilized/removed and all the players are informed.

15. The program will take care of the exit-mechanic when the Rats complete the necessary puzzles.
    - Description: Rats are required to touch a number of certain objects in a certain order or even at the same time to unlock an exit.
    - Precondition: The Rats have satisfied the requirements in the puzzle.
    - Postcondition: A door of escape is opened and the Rats can use it to win.

16. The application will pronounce the Rats the victors in case they get out of the house before the timer is over.
    - Description: The Rats win in case any of the Rats takes the escape route which was activated before the time elapses.
    - Precondition: A Rat interacts with an escape mechanism that is active.
    - Post condition: A game is over, and a result of Rats Win appears.

17. The application will indicate that the Cat was a winner when all the Rats are caught before the timer runs out.
    - Description: The Cat is a winner when all Rat players are caught before time elapses or when the Rats evade.
    - Precondition: The population of active Rats is zero.

- Post condition: The game finishes and a result of the game is shown as Cat Wins.

18. The app will show the outcomes of the game and update statistics of the players.
    - Description: When a game is over, it will display a summary screen that displays the result (win/loss) and player statistics (e.g. wins, losses).
    - Precondition: There is a win/loss condition that has been achieved.
    - Result: The results are displayed to players, and they are taken back to the lobby.

19. The app will enable one to log out.
    - Description: An authenticated user is able to terminate his/her session in a secure way.
    - Prerequisite: The user should be logged-in.
    - Postcondition: The session of the user is terminated, and one is redirected to the main landing page.
    -

20. The application will give an audio and custom control menu.
    - Description: The user is able to customize the volume of music and sound effects and customize control keys (possibly).
    - Precondition: The user is either in the main menu or in the lobby.
    - Postcondition: Preferences of user are stored and implemented.

3. SYSTEM ARCHITECTURE

The system follows a client-server model to ensure fair and synchronized gameplay.

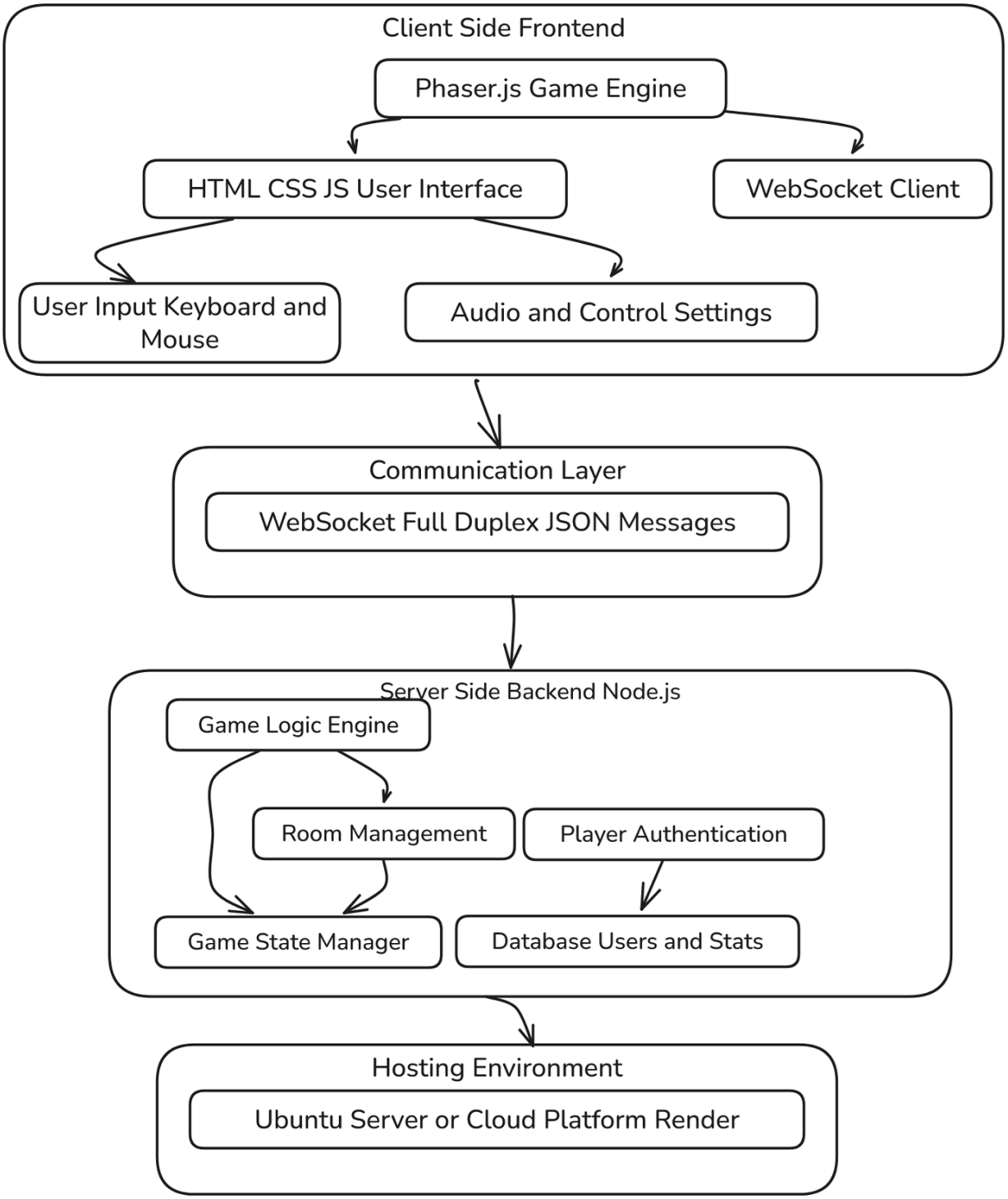
Client-Side (Frontend): Built with Phaser.js, HTML, CSS, and JavaScript. Responsible for rendering graphics, playing audio, capturing user input, and communicating with the server.

Server-Side (Backend): Built with Node.js. Acts as the authoritative host, managing game logic, state, player authentication, and room management.

Communication: A persistent, full-duplex WebSocket connection is maintained between each client and the server for real-time, low-latency data exchange using JSON messages.

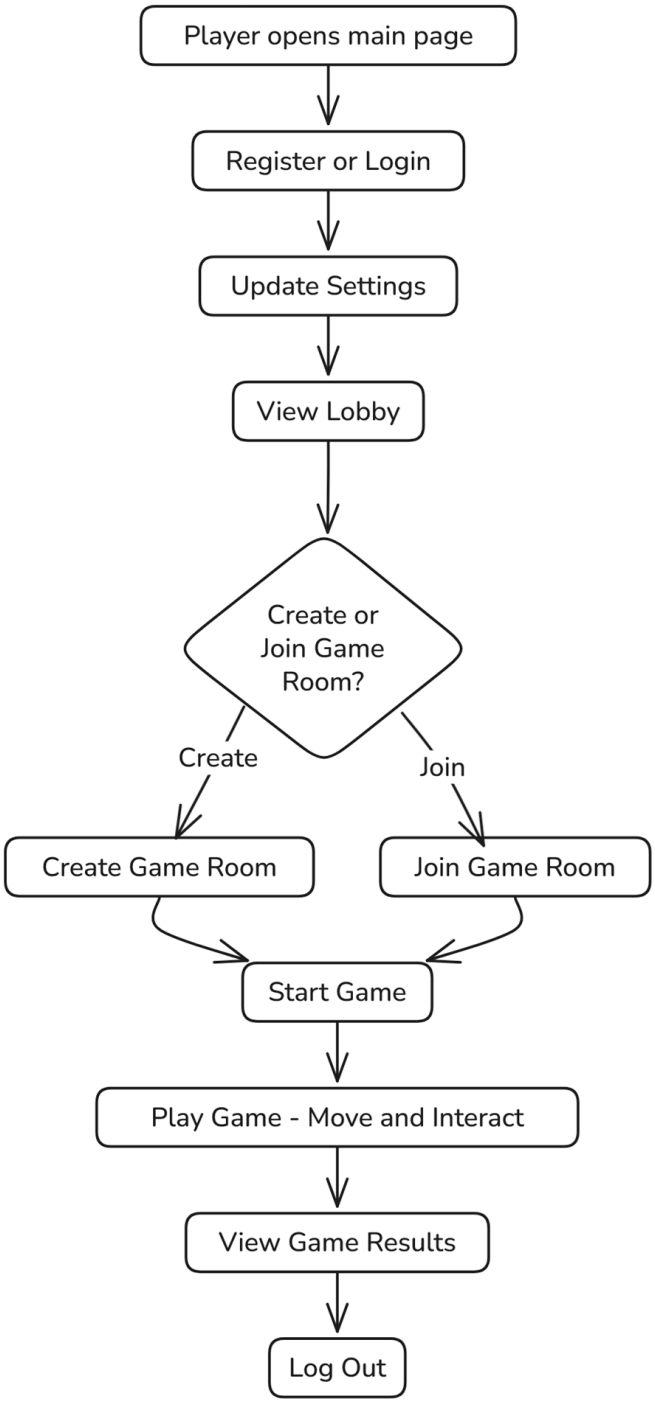
Hosting: The server can be deployed on a dedicated Ubuntu instance or a cloud platform like Render.

Figure 3.1: Client-Server Architecture Diagram



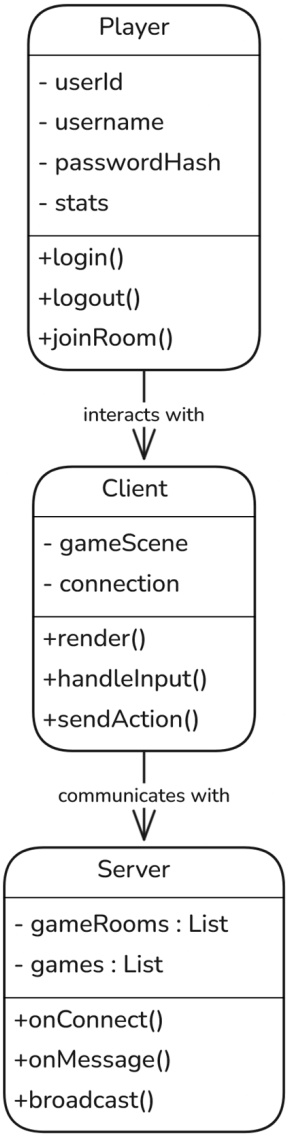
4. UML DIAGRAMS

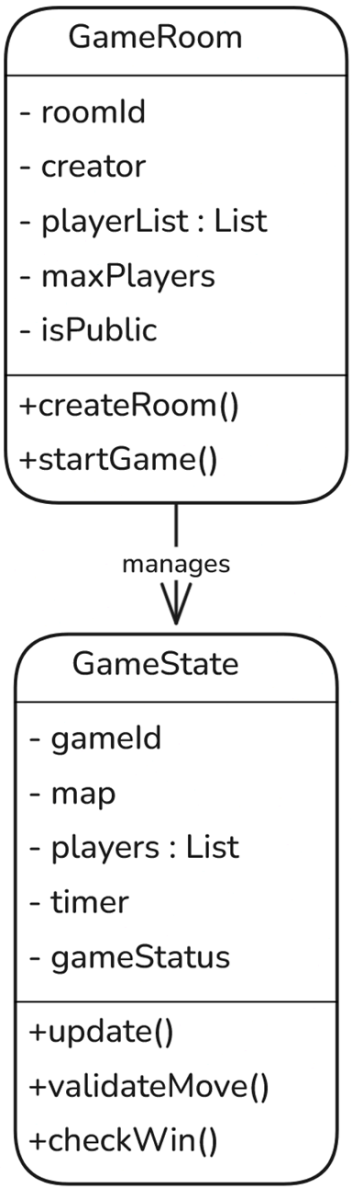
Figure 4.1: Use Case Diagram for Rat Race Escape

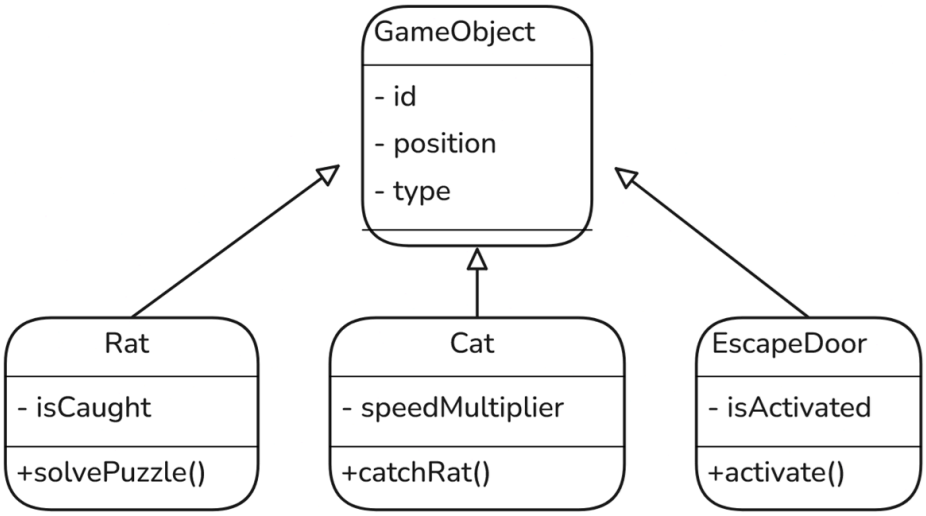


4.2 Class Diagram (Simplified Core Classes)

Figure 4.2: Simplified Class Diagram for Rat Race Escape

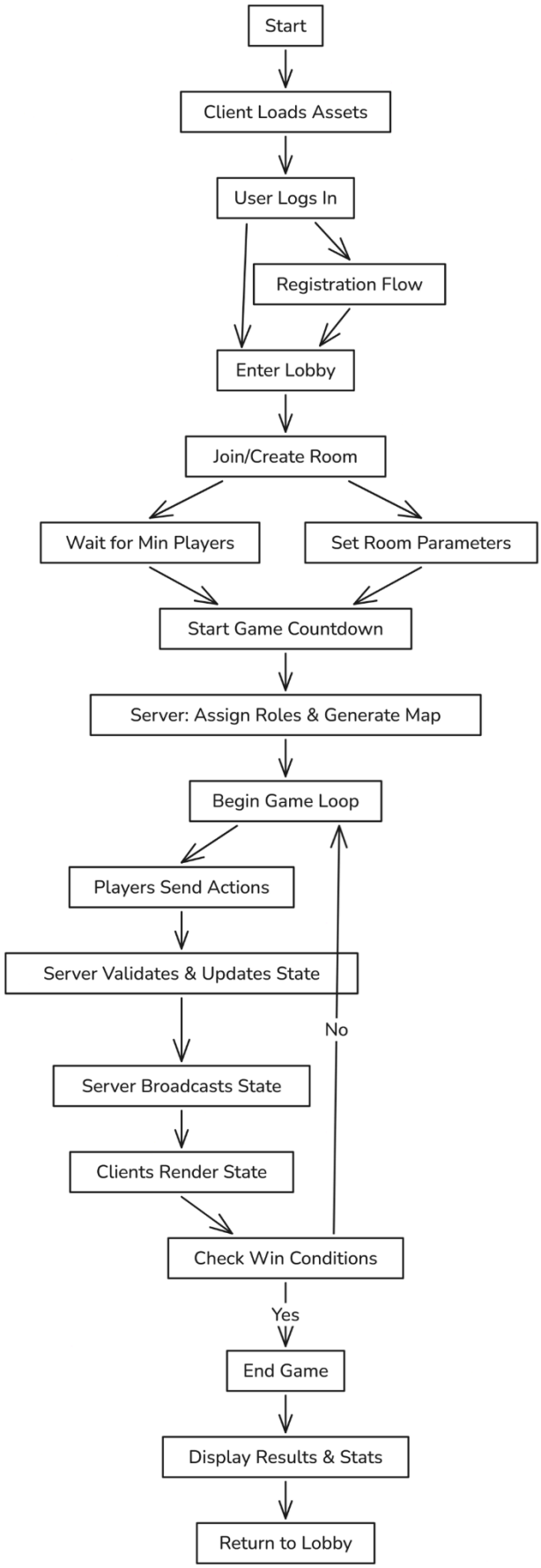



GameRoom
- roomId
- creator
- playerList : List
- maxPlayers
- isPublic
+createRoom()
+startGame()
manages
GameState
- gameId
- map
- players : List
- timer
- gameStatus
+update()
+validateMove()
+checkWin()


GameObject
- id
- position
- type
Rat
- isCaught
+solvePuzzle()
Cat
- speedMultiplier
+catchRat()
EscapeDoor
- isActivated
+activate()

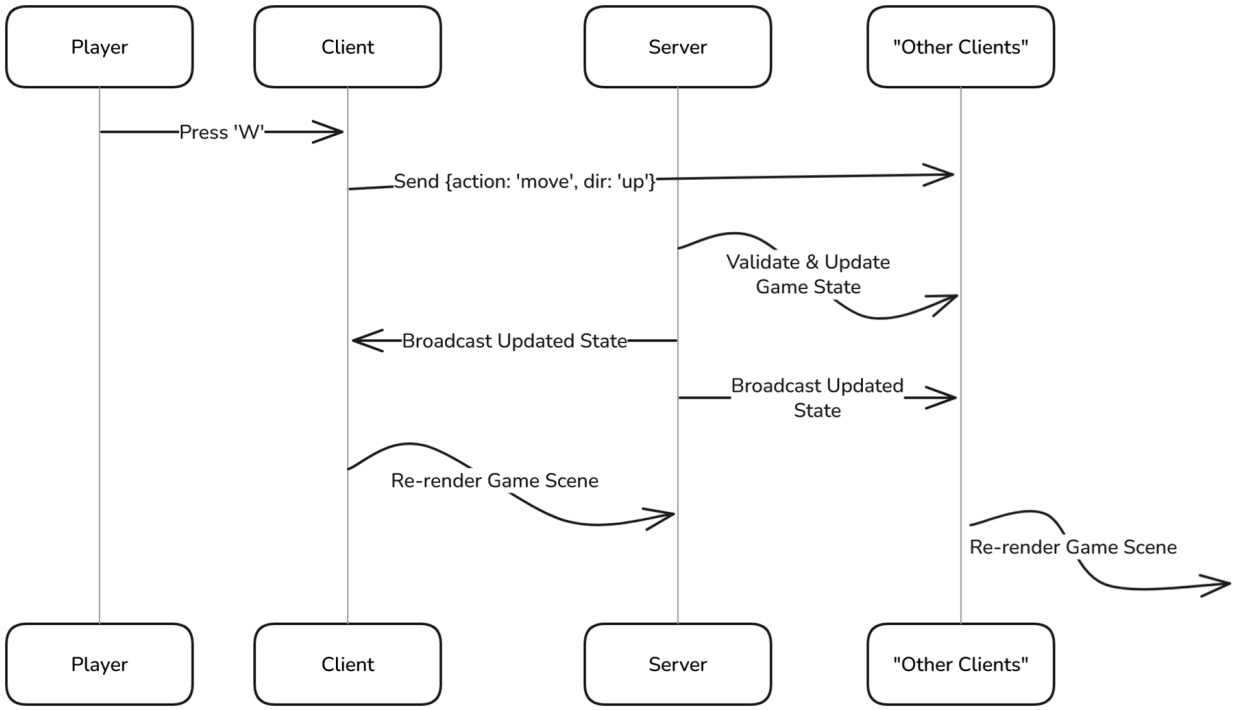
4.3 Activity Diagram (High-Level Game Flow)

Figure 4.3: High-Level Game Flow Activity Diagram


Start
Client Loads Assets
User Logs In
Registration Flow
Enter Lobby
Join/Create Room
Wait for Min Players
Set Room Parameters
Start Game Countdown
Server: Assign Roles & Generate Map
Begin Game Loop
Players Send Actions
Server Validates & Updates State
No
Server Broadcasts State
Clients Render State
Check Win Conditions
Yes
End Game
Display Results & Stats
Return to Lobby

4.4 Sequence Diagram (Player Movement and State Update)

Figure 4.4: Sequence Diagram for Player Movement and State Update



This sequence ensures that all players see a synchronized view of the game world, with the server as the single source of truth.